

note-gen-ai

Microservices Architecture & Full Product Flow

Goal: A web application that records lectures, captures microphone/display audio and video, transcribes speech live, lets the user choose an AI template, generates structured notes with Gemini AI, and stores those notes into user-managed folders.

Primary stack: React + TypeScript, Java + Spring Boot, Python + FastAPI, Faster-Whisper, Gemini AI, PostgreSQL, MongoDB, Redis, RabbitMQ, S3/MinIO, Docker and Kubernetes.

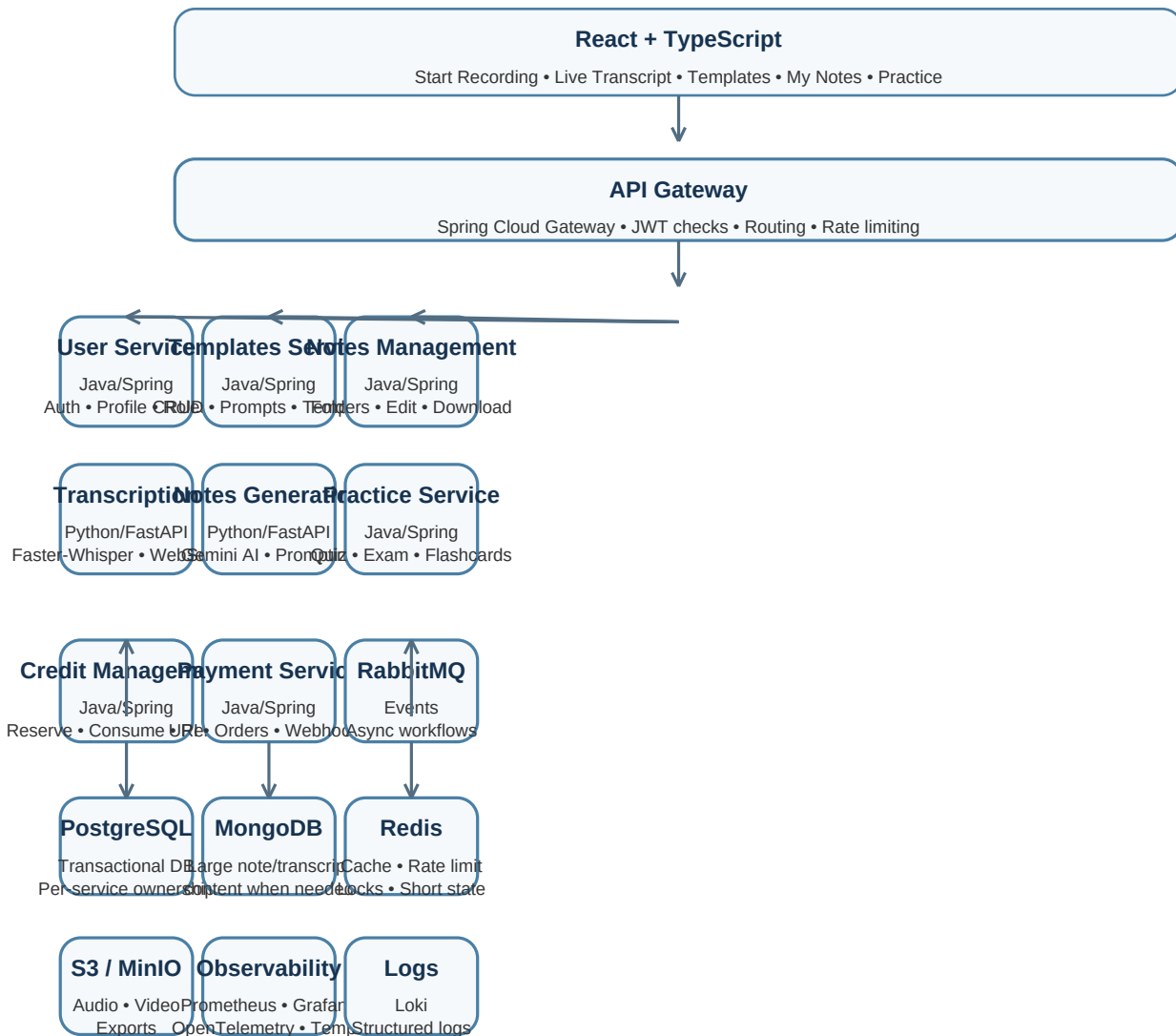
1. Full User Flow

Step	User action	System behavior
1	Open application	Show Start Recording screen.
2	Click Start Recording	Request microphone and display/screen permissions.
3	Grant permissions	Capture microphone audio, display/video and available system audio.
4	Recording continues	Send chunks to Transcription Service and show live transcript.
5	Stop Recording	Finalize recording and transcript, then show Templates.
6	Select template	Send transcript + template to Notes Generation Service.
7	Generate	Gemini AI produces structured notes.
8	Show result	Display generated note in the UI.
9	Save	User chooses a folder in My Notes and stores the note.

2. Full Architecture Diagram

The frontend is the only browser-facing application. The API Gateway is the normal entry point for REST APIs. Long-running AI/audio workflows use RabbitMQ events.

note-gen-ai Microservices Architecture



3. Service Responsibilities

Service	Technology	Main responsibility
Transcription	Python / FastAPI	Audio/video processing, Faster-Whisper transcription, live transcript
Notes Generation	Python / FastAPI	Gemini AI prompts and structured note generation
Templates	Java / Spring Boot	Template CRUD, prompt definitions, system/user templates
Notes Management	Java / Spring Boot	My Notes, folders, editing, download/export, versions
Practice	Java / Spring Boot	Reading, MCQ, exams, flashcards, scoring, progress
Credit Management	Java / Spring Boot	Credit balance, reserve, consume, release, history
Payment	Java / Spring Boot	UPI orders, provider integration, verification, webhooks
User Management	Java / Spring Boot	Registration, authentication, profile, roles
API Gateway	Java / Spring Cloud	Routing, auth checks, rate limiting, API entry point

4. Technologies Required

Area	Technologies
Frontend	React, TypeScript, Vite, React Router, TanStack Query, Zustand, Axios
Browser capture	MediaRecorder API, getUserMedia, getDisplayMedia, Web Audio API, WebSocket
Java	Java 21+, Spring Boot, Spring Security, Spring Data JPA, Hibernate, Spring Cloud Gateway
Python	Python 3.12+, FastAPI, Uvicorn, Pydantic, Faster-Whisper, FFmpeg
AI	Google Gemini AI; optional Ollama/local LLM abstraction
Databases	PostgreSQL, MongoDB
Cache	Redis
Messaging	RabbitMQ
File storage	S3 or MinIO
Observability	Prometheus, Grafana, OpenTelemetry, Tempo, Loki
Deployment	Docker, Docker Compose, Kubernetes
CI/CD	GitHub Actions, container registry
Testing	JUnit 5, Mockito, Testcontainers, pytest, React Testing Library

5. Detailed Recording and AI Flow

```
React UI
|
| Start Recording
v
Request microphone + display permissions
|
v
MediaRecorder + Web Audio
|
+---- audio chunks -----+
| |
v v
Transcription Service Object Storage
Python/FastAPI S3 / MinIO
|
+---- WebSocket transcript -----> React UI
|
v
Final Transcript
|
v
Stop Recording
|
v
Templates Service
|
v
User selects template
|
v
Notes Generation Service
Python/FastAPI
|
v
Gemini AI
|
v
Structured Note
|
v
React Note Editor
|
v
Notes Management Service
|
v
Selected Folder in My Notes
```

Browser capture note

Microphone capture is supported through the browser permission flow. Display capture can include video and, depending on browser/OS/source selection, system audio. The UI must handle the case where the selected display source does not provide system audio.

6. My Notes

My Notes is responsible for note lifecycle management: folders, editing, rename, move, delete, favorite, archive, search, version history, download/export and optional regeneration using another template.

```
My Notes
|
+-- Folder: Java
| +-- Spring Boot Basics
| +-- Dependency Injection
|
+-- Folder: College
+-- Folder: Work
+-- Folder: Personal
```

7. Practice Flow

```
Saved Note
|
v
```

```
Practice Service
|
+--> Read mode
+--> MCQ
+--> True/False
+--> Flashcards
+--> Mock Exam
|
v
Answer
|
v
Score + Progress + History
```

8. Credit and Payment Architecture

```
AI Request
|
v
Credit Management
|
+-- Check balance
|
+-- Reserve credits
|
v
Notes Generation
|
+-----+-----+
| |
Success Failure
| |
v v
Consume Release
credits credits
```

UPI payment flow

```
React
|
v
API Gateway
|
v
Payment Service
|
v
Payment Provider / UPI
|
v
Provider Webhook
|
v
Payment Service verifies webhook
|
v
PaymentCompleted event
|
v
Credit Management adds credits
```

9. Event-Driven Architecture

RabbitMQ should be used for workflows that do not need an immediate synchronous response. Examples include transcription completion, AI generation, payment completion and credit updates.

```
RecordingUploaded
TranscriptionStarted
TranscriptionCompleted
TranscriptionFailed

NoteGenerationRequested
NoteGenerationStarted
NoteGenerationCompleted
NoteGenerationFailed

CreditsReserved
CreditsConsumed
CreditsReleased

PaymentCreated
PaymentCompleted
PaymentFailed

PracticeStarted
PracticeCompleted
```

10. Data Ownership

```
user-service
-> user_db
```

```
templates-service
-> template_db

notes-management-service
-> notes_db

practice-service
-> practice_db

payment-service
-> payment_db

credit-management-service
-> credit_db

transcription / notes generation
-> service-owned job/transcript data

Shared infrastructure:
PostgreSQL | MongoDB | Redis | RabbitMQ | S3/MinIO
```

11. Recommended Repository Structure

```
note-gen-ai/  
|  
+-- frontend/  
|   +-- note-gen-ui/  
|   |  
+-- services/  
|   +-- api-gateway/  
|   +-- user-service/  
|   +-- transcription-service/  
|   +-- notes-gen-service/  
|   +-- templates-service/  
|   +-- notes-management-service/  
|   +-- practice-service/  
|   +-- payment-service/  
|   +-- credit-management-service/  
|   |  
+-- infrastructure/  
|   +-- docker/  
|   +-- postgres/  
|   +-- mongodb/  
|   +-- redis/  
|   +-- rabbitmq/  
|   +-- prometheus/  
|   +-- grafana/  
|   +-- tempo/  
|   +-- loki/  
|   +-- kubernetes/  
|   |  
+-- docs/  
|   +-- architecture/  
|   +-- api/  
|   +-- events/  
|   +-- database/  
|   |  
+-- .github/  
|   +-- workflows/  
|   |  
+-- README.md
```

12. Recommended Service Ports

Component	Port
React UI	3000
API Gateway	8080
User Service	8081
Transcription Service	8082
Notes Generation	8083
Templates Service	8084
Notes Management	8085
Practice Service	8086
Payment Service	8087
Credit Management	8088
PostgreSQL	5432
MongoDB	27017
Redis	6379
RabbitMQ	5672
RabbitMQ Management	15672
Prometheus	9090
Grafana	3001

13. Development Roadmap

Phase	Deliverables
1 - Core	React UI, User Service, Gateway, Transcription, Notes Generation, Notes Management
2 - Organization	Templates, folders, editing, search, export/download
3 - Learning	Practice, MCQ, exams, flashcards, progress
4 - Monetization	Credit Management, UPI Payment, subscriptions/packages
5 - Production	RabbitMQ, Redis, S3/MinIO, observability, Docker, Kubernetes, CI/CD

14. Key Architecture Rules

- Each microservice owns one business domain and its data.
- Use REST for immediate operations and RabbitMQ for asynchronous workflows.
- Keep audio/LLM workloads in Python and business/transaction workloads in Java.
- Never let the frontend decide payment success or credit balance.
- Reserve credits before expensive AI generation and consume/release after the result.
- Store audio/video in object storage rather than relational database blobs.
- Keep Gemini behind an AI provider abstraction so another model can be added later.
- Use WebSocket for near-live transcript delivery.
- Add distributed tracing, metrics and structured logs from the beginning.